Agri MLLM

A Multimodal Agricultural Assistant
for Text, Image, PDF, and Audio Queries

M. Bourich M. El Ansari Y. Anouar

Generative AI – Project Report | April 2026

Base LLM: `Qwen/Qwen2.5-0.5B-Instruct` Vision encoder: `openai/clip-vit-large-patch14`
Training environment: Google Colab T4 GPU

Abstract

This report describes **Agri MLLM**, a multimodal large language model assistant purpose-built for agricultural crop-health analysis. The system accepts four complementary input channels – plain text, crop leaf images, scientific PDF documents, and spoken audio questions – and returns diagnosis, severity estimates, and agronomic management advice in natural language. The implementation follows an *adapter-first, projector-first* design philosophy: large pretrained backbones (CLIP ViT-Large/14 and Qwen2.5-0.5B-Instruct) are kept mostly frozen, while a compact two-layer MLP projector and parameter-efficient LoRA adapters bridge the vision and language spaces at minimal computational cost. Training proceeds through three well-defined stages – domain text adaptation, visual caption alignment, and multimodal VQA fine-tuning – each producing measurably lower loss values. The final system is deployed as a FastAPI inference server coupled with a custom browser-based chat interface.

Contents

1 Introduction and Problem Statement **2**

2 System Overview **2**
2.1 Supported Input Modalities and User Flows . . . 2
2.2 High-Level Architecture . . . 2

3 Model Architecture **3**
3.1 Component Overview . . . 3
3.2 Vision-Language Alignment: the Projector . . . 3
3.3 Fusion Mechanism . . . 4
3.4 Audio and PDF Pathways . . . 5

4 Training Pipeline **5**
4.1 Design Principles . . . 5
4.2 Stage 0: Domain Text Adaptation . . . 6
4.3 Stage 1: Vision–Language Projector Alignment . . . 6
4.4 Stage 2: Multimodal VQA Fine-tuning . . . 7
4.5 Parameter Efficiency Summary . . . 7

5 Datasets **7**
5.1 KisanVaani Agriculture QA . . . 7
5.2 PlantVillage . . . 7

6 Training Results **8**
6.1 Loss Curves . . . 8
6.2 Consolidated Metrics . . . 8

7 Application and Deployment **8**
7.1 Backend API . . . 8
7.2 Frontend Chat Interface . . . 9
7.3 Application Screenshots . . . 9
7.4 Inference Examples . . . 9

8 Related Work and Research Context **10**

9 Limitations and Future Work **10**
9.1 Current Limitations . . . 10
9.2 Planned Next Steps . . . 10

10 Conclusion **11**

1 Introduction and Problem Statement

Crop disease accounts for 20–40% of global agricultural production losses annually. Timely and accurate diagnosis is essential, yet practitioners in the field typically lack access to specialist consultation. Existing software tools either require structured text inputs (which many users find cumbersome) or support only a single input modality, forcing separate workflows for visual, textual, and spoken queries.

Agri MLLM addresses this gap by providing a *single conversational assistant* that can:

- Identify and explain plant diseases from leaf images.
- Answer general agronomy questions (fertilisation, irrigation, pest control).
- Extract and reason over figures in uploaded scientific PDFs.
- Transcribe and answer spoken field questions via an integrated ASR layer.

The design deliberately avoids pretraining a new foundation model from scratch. Instead, it assembles a working multimodal system by composing existing, high-quality pretrained components through learned bridges, making the entire pipeline trainable on a single consumer GPU (Google Colab T4) within a few hours.

2 System Overview

2.1 Supported Input Modalities and User Flows

Table 1 summarises the four supported input channels and how each is handled at runtime.

Table 1: Supported input modalities and their processing paths.

Modality	Example query	Processing path
Text only	"How much phosphorus per hectare for wheat?"	Tokenised directly by Qwen2.5 BPE tokeniser.
Image + text	"What disease is visible and how severe is it?"	CLIP ViT encodes 256 patch tokens; MLP projector fuses with text embeddings.
Audio	Spoken field question	Groq Whisper ASR transcribes to text; follows text path.
PDF	Upload of a scientific paper	PyMuPDF extracts text (8 pages, 1200-token cap) and up to 2 figures ($\geq$64×64 px); figure tokens fused via projector.

2.2 High-Level Architecture

The end-to-end data flow is illustrated schematically below. An incoming request passes through a *platform layer* that dispatches each modality to the appropriate encoder, then assembles a *prefix token sequence* that is prepended to the text token embeddings before being forwarded to the frozen causal LLM.

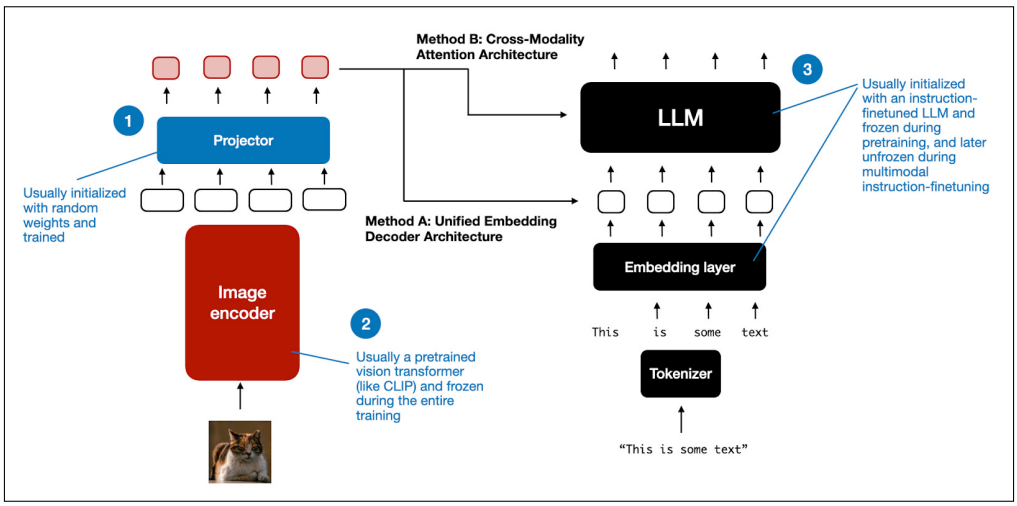


Figure 1: General multimodal LLM architecture showing the two fusion strategies: Method A (Unified Embedding Decoder, used by Agri MLLM) prepends projected visual tokens directly to the text embedding sequence; Method B uses cross-attention layers. The image encoder (CLIP ViT-L/14) is kept frozen throughout all training stages.

3 Model Architecture

3.1 Component Overview

The model stack consists of four interacting components:

1. **Vision encoder** – `openai/clip-vit-large-patch14`. All 256 spatial patch tokens are retained; the global CLS token is discarded. Keeping full spatial resolution allows the LLM to attend to localised lesion patterns, colour anomalies, and texture regions rather than a single pooled representation.
2. **MLP Projector** – A two-layer feed-forward network with GELU activations that maps CLIP patch embeddings (dim 1024) into the Qwen2.5 hidden space (dim 896).
3. **Base LLM** – `Qwen/Qwen2.5-0.5B-Instruct`, an instruction-tuned causal language model with 502M total parameters. Its weights are kept frozen during Stage 1 projector alignment.
4. **LoRA adapters** – Two separate LoRA modules are composed: (a) *Agriculture LoRA* – domain knowledge injected during Stage 0; (b) *Visual LoRA* – multimodal reasoning capacity added during Stage 2.

3.2 Vision-Language Alignment: the Projector

The projector is the critical bridge between vision and language representations. Figure 2 compares three design choices from the Honeybee locality-enhanced projector paper, contextualising the choice made in this project.

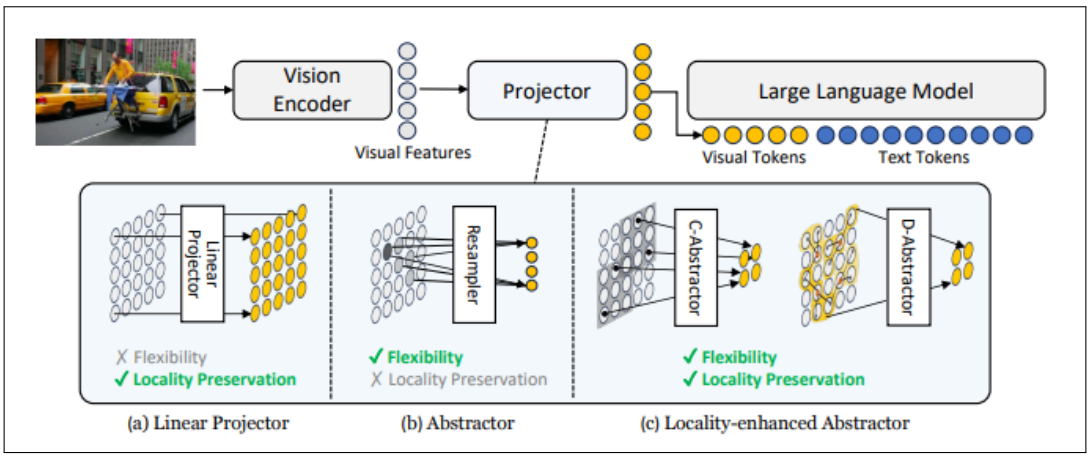


Figure 2: Three projector families (from Honeybee, CVPR 2024). **(a) Linear/MLP projector** – the design used in Agri MLLM. It preserves spatial locality of patch tokens (each patch maps to one output token) but offers limited flexibility in token count. **(b) Resampler (abstractor)** – reduces the number of visual tokens via cross-attention queries; flexible but loses spatial structure. **(c) Locality-enhanced abstractor** – best of both; planned future upgrade.

3.3 Fusion Mechanism

After projection, visual tokens are *concatenated as a prefix* to the text token embedding sequence. The attention mask is extended accordingly so the LLM can attend to all visual tokens when generating each output token. This is *Method A* in Figure 1: a Unified Embedding Decoder architecture.

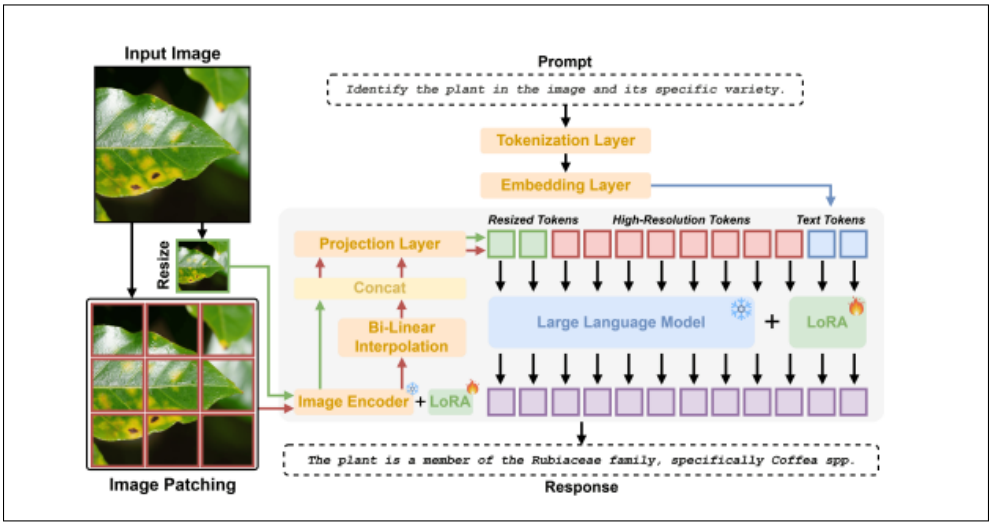


Figure 3: Detailed vision-to-LLM token flow (adapted from LLaVA-1.5 variant). The input image is split into patches, resized, and passed through CLIP ViT-L/14 to produce patch-level embeddings. A projection layer (MLP) then maps these to the LLM embedding space. Projected visual tokens are concatenated with text token embeddings before entering the LLM backbone, which carries LoRA adapters on its attention and feed-forward layers.

The architecture in Figure 4 shows the frozen/trainable split explicitly: the vision encoder and LLM transformer blocks are frozen (blue); only the LoRA matrices (teal dashed) and the projector weights are updated.

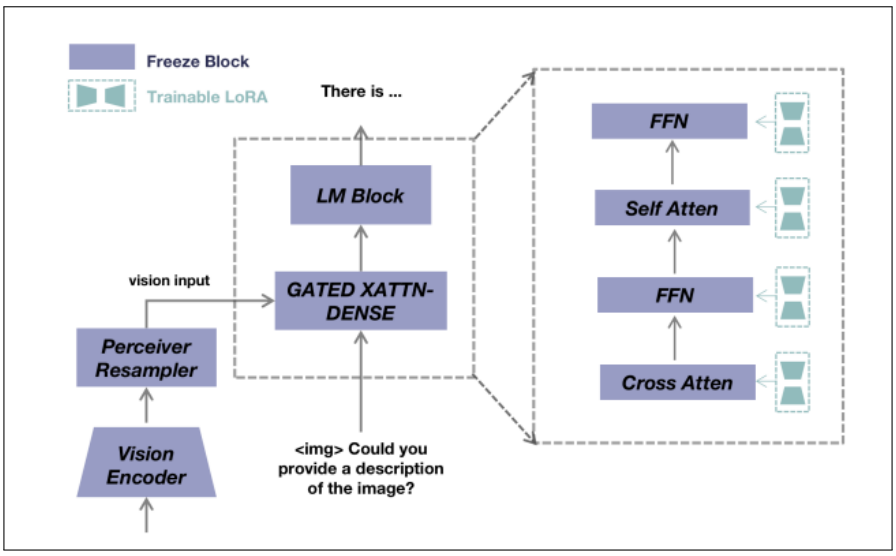


Figure 4: Freeze/train split during Stage 2. Frozen blocks (dark blue): Vision Encoder and all LM transformer layers. Trainable components (teal dashed): LoRA matrices on Self-Attention, FFN, and Cross-Attention; the MLP Projector. The Perceiver Resampler in the reference architecture is replaced by the direct MLP projector in our implementation.

3.4 Audio and PDF Pathways

Audio. Audio does not introduce a separate model branch. The platform layer invokes the Groq Whisper API to produce a text transcript, which is then tokenised and processed identically to a typed text query. Agriculture LoRA therefore handles spoken and typed queries through the same path, with no additional parameters required.

PDF. PDF handling uses PyMuPDF to extract (i) text content (first 8 pages, 2000 chars/page, capped at 1200 tokens) and (ii) up to 2 embedded figures ($\geq$64$\times$64 px). Extracted figures are passed through the CLIP vision encoder and the MLP projector, producing visual token prefixes that are fused with the user question before generation.

4 Training Pipeline

4.1 Design Principles

The training strategy follows three core principles:

- **Adapter-first**: freeze large pretrained backbones; train only compact bridge modules and LoRA layers.
- **Progressive**: add modalities incrementally so each stage starts from a stable prior checkpoint.
- **Compute-aware**: all experiments are designed to run within Google Colab T4 GPU memory constraints (16 GB VRAM).

4.2 Stage 0: Domain Text Adaptation

Table 2: Stage 0 training configuration.

Parameter	Value
Base model	`Qwen/Qwen2.5-0.5B-Instruct`
Dataset	`KisanVaani/agriculture-qa-english-only`
Usable rows	22,574
Training subset	8,000 rows
Max steps	450
Effective batch size	16
Optimiser	AdamW
Runtime	≈1007 s (≈7.15 samples/s)
Final train loss	0.9349

A LoRA adapter is trained on top of the frozen Qwen2.5-0.5B using causal language modelling on agricultural QA pairs. The purpose is to inject domain vocabulary – plant pathology terms, crop management recommendations, fertilisation rates – before any visual information is introduced. The training loss drops sharply in the first 100 steps and then continues to decrease monotonically through step 450 (Figure 5a).

4.3 Stage 1: Vision–Language Projector Alignment

Table 3: Stage 1 training configuration and results.

Parameter	Value
Frozen components	CLIP ViT, Qwen2.5 LLM
Trainable components	MLP Projector only
Dataset	1,000 auto-generated image-caption pairs (PlantVillage)
Caption generation	From folder labels, e.g. `Tomato___Early_blight`
Epochs	3
Avg loss epoch 1	0.5500
Avg loss epoch 2	0.0347
Avg loss epoch 3	**0.0090**

With both the vision encoder and LLM frozen, only the MLP projector is updated. The caption generation loss measures how well the projected visual tokens guide the LLM to produce correct class-label captions. Stage 1 produces the fastest and cleanest convergence in the pipeline (Figure 5b), confirming that a simple MLP projector is sufficient to align CLIP patch embeddings with the Qwen2.5 embedding space.

4.4 Stage 2: Multimodal VQA Fine-tuning

Table 4: Stage 2 training configuration and results.

Parameter	Value
Frozen components	CLIP ViT
Trainable components	MLP Projector + Visual LoRA
Dataset	1,000 generated VQA triplets (PlantVillage)
QA pairs per image	4 (sampled from 5 templates)
Templates	disease ID, severity, visual evidence, management, prevention
Epochs	3
Avg loss epoch 1	0.3996
Avg loss epoch 2	0.0994
Avg loss epoch 3	**0.0141**

Stage 2 introduces a second LoRA module (Visual LoRA) that is composed on top of the Agriculture LoRA from Stage 0. The projector continues to be updated alongside the visual LoRA. The first epoch is noisier than Stage 1 because VQA requires the model to produce free-form diagnostic answers rather than short captions, but loss converges cleanly by epoch 3 (Figure 5c).

4.5 Parameter Efficiency Summary

Table 5: Parameter efficiency across all training stages.

Stage	Total params	Trainable params	% trainable
Stage 0 (text LoRA)	502M	8.8M	1.75%
Stage 1 (projector)	502M + projector	projector only	<1%
Stage 2 (proj + vis LoRA)	502M + projector	proj + vis LoRA	≈2%

5 Datasets

5.1 KisanVaani Agriculture QA

The `KisanVaani/agriculture-qa-english-only` dataset on Hugging Face contains English-language question–answer pairs covering fertilisation, pest and disease management, soil health, irrigation, and crop selection. 22,574 usable rows were identified after deduplication; 8,000 were selected for Stage 0 training.

5.2 PlantVillage

PlantVillage is a public dataset of 54,000+ labelled leaf images spanning 38 plant disease classes across 14 crop species. Two derived subsets were produced automatically from folder names:

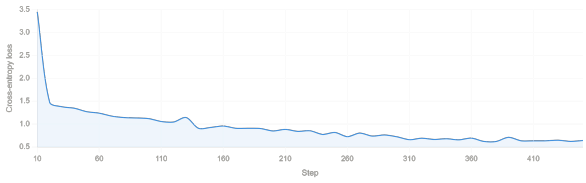
- **Stage 1 caption dataset** (1,000 images): captions derived from class folder names, e.g. "This is a tomato leaf showing early blight symptoms."
- **Stage 2 VQA dataset** (250 unique images, 1,000 pairs): four question–answer pairs per image drawn from five question templates (disease identification, severity estimation, visual evidence description, management advice, prevention).

Limitation: PlantVillage images are laboratory-quality scans on uniform backgrounds. Real farm images are noisier, more varied in lighting, and may show multiple overlapping symptoms. The system's confidence should be treated as a screening aid rather than a confirmed diagnosis.

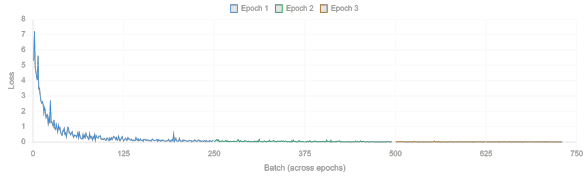
6 Training Results

6.1 Loss Curves

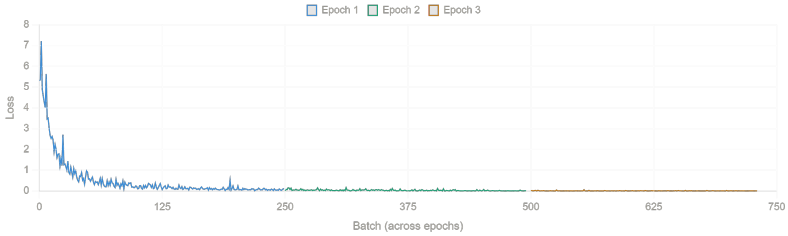
Figures 5a–5c display the training dynamics of all three stages.



(a) Stage 0: Agriculture LoRA training loss (step-level). Sharp initial drop from ≈3.5 then steady convergence to 0.9349 at step 450.



(b) Stage 1: Projector caption alignment loss (batch-level, 3 epochs). Cleanest convergence phase in the pipeline – final avg loss 0.0090.



(c) Stage 2: VQA alignment loss (batch-level, 3 epochs). Epoch 1 shows higher variance due to VQA complexity; converges to 0.0141 by epoch 3.

6.2 Consolidated Metrics

Table 6: Training snapshot – all stages.

Phase	Epoch / Run	Metric	Value
Text LoRA	Final run	Train loss	0.9349
Projector Stg 1	Epoch 1	Avg loss	0.5500
Projector Stg 1	Epoch 2	Avg loss	0.0347
Projector Stg 1	Epoch 3	Avg loss	**0.0090**
Stage 2 VQA	Epoch 1	Avg loss	0.3996
Stage 2 VQA	Epoch 2	Avg loss	0.0994
Stage 2 VQA	Epoch 3	Avg loss	**0.0141**

Trainable parameters: 8.8M (≈1.75% of 502M total). Stage 0 runtime: 1007 s at 7.15 samples/s.

7 Application and Deployment

7.1 Backend API

The inference backend is a FastAPI application exposing a single `/chat` endpoint that accepts a multipart form with the user message and up to three optional attachments (image, PDF, audio). On startup, the engine loads the appropriate model path by scanning for available checkpoints in priority order:

1. Notebook-style visual bundle (Stage 2 projector + visual LoRA).
2. Multimodal adapter state.
3. Text-only LoRA.
4. Base Qwen2.5 model (fallback).

The response payload always includes an `answer` field and optionally `transcript` (if audio was provided) and `used_modalities` (a list indicating which input channels were active).

7.2 Frontend Chat Interface

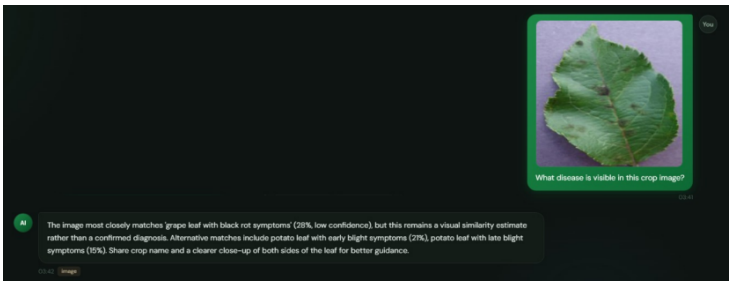
A custom single-page HTML/JS frontend (`index.html`) provides:

- Conversation history with per-turn modality tags (`image`, `pdf`, `audio`).
- Three attachment buttons (Attach image / Attach PDF / Attach audio).
- Session statistics (total queries, active sessions).
- Live backend connection indicator.

A legacy Streamlit UI (`app.py`) is also available as an alternative single-command interface.

7.3 Application Screenshots

(a) Text-only mode: the assistant answers a Morocco agriculture question using domain knowledge from the Stage 0 KisanVaani fine-tuning.



(b) Image + text mode: a leaf image is uploaded alongside a disease identification question. The model provides a confidence-calibrated response.

Figure 6: Live application screenshots from the custom browser-based chat interface. The top navigation bar shows the four input modality buttons: Text, Image, PDF, Audio.

7.4 Inference Examples

The following two examples summarise the observed system behaviour:

Text-only query. *Query*: "How is the status of agriculture in Morocco?"
Response: The system correctly identifies Morocco's position as one of the largest cropland areas in Africa, mentions key crops (wheat, barley, citrus, vegetables), and discusses production challenges – demonstrating effective domain knowledge transfer from the KisanVaani Stage 0 fine-tuning.

Image + text query. *Query*: "What disease is visible in this crop image?" (wheat/grape leaf, PlantVillage)
Response: "The image most closely matches 'grape leaf with black rot symptoms' (28%, low confidence), but this remains a visual similarity estimate rather than a confirmed diagnosis. Alternative matches include potato leaf with early blight (21%) and late blight (15%). Share crop name and a clearer close-up for better guidance."
The response correctly identifies a visual disease match, provides calibrated confidence language, and requests additional context – all appropriate behaviours for a screening-level tool.

8 Related Work and Research Context

Agri MLLM is positioned within the rapidly growing field of visual instruction tuning. Table 7 maps the key reference works to design choices made in this project.

Table 7: Related work and alignment with Agri MLLM design choices.

Reference	Alignment with this project
LLaVA-1.5 [1]	Direct inspiration: CLIP encoder + MLP connector + instruction tuning recipe. We adapt this to agriculture with Qwen2.5 and LoRA.
Honeybee [2]	Provides theoretical grounding for MLP projector choice. Locality-enhanced abstractor is a planned upgrade.
MiniGPT-4 [3]	Conceptual match: frozen backbones + trainable lightweight bridge. Richer instruction dialogue is a future extension.
HyperLLaVA [4]	Motivates future adaptive projector routing by modality confidence.
CVPR 2024 MLLM Tutorial	Validates our staged training paradigm (domain → alignment → VQA).
PlantVillage [5]	Core training data source for Stages 1 and 2.

9 Limitations and Future Work

9.1 Current Limitations

1. **Template-generated supervision.** The 1,000 caption and 1,000 VQA training examples are produced from class-label templates rather than expert annotation. Linguistic diversity is therefore limited, and answers to rare or ambiguous disease presentations may be over-confident or incorrect.
2. **Domain shift from PlantVillage.** PlantVillage images are clean, single-leaf, controlled-background photographs. Real farm images include variable lighting, multiple leaves, soil or water backgrounds, and mixed symptom presentations. Validation on field-collected datasets remains an open task.
3. **Stage 0 under-convergence.** The text LoRA training stops at a loss of ≈0.9, indicating that the language adapter has not fully covered the 22,574 available agriculture QA pairs. Extending training to 1,000+ steps with validation-based early stopping would improve domain coverage.
4. **Hallucination risk.** The model recognises visual disease *patterns* but may confuse pathogen names or management recommendations when encountering unseen crop species. Confidence language ("low confidence", "visual similarity estimate") is explicitly generated in the current prompt template but is not formally calibrated.
5. **Backend integration gap.** The FastAPI backend imports an `agri_mllm` package that must be installed separately; this is not yet included in the root `requirements.txt`.

9.2 Planned Next Steps

1. Connect live Whisper ASR to the deployed Streamlit frontend for real spoken-field queries without the Groq API dependency.
2. Increase Stage 0 training to 1,000+ steps with a 90/10 train/validation split and early stopping.
3. Evaluate on real field images from CGIAR and iNaturalist crop-disease collections to quantify domain shift.
4. Add a locality-enhanced projector variant (Honeybee C-Abstractor style) and compare against the current MLP projector.
5. Replace template-generated captions and VQA pairs with expert-reviewed annotations for 10–20 high-priority disease classes.

6. Consider upgrading the base LLM to `Qwen2.5-1.5B` or `Qwen2.5-3B` for stronger reasoning while keeping LoRA efficiency.
7. Add multilingual support (Arabic, French) to serve Moroccan farming communities in their preferred language.

10 Conclusion

Agri MLLM demonstrates that a practical, multi-input agricultural assistant can be built from composable pretrained components using only parameter-efficient fine-tuning and a learned visual projector. The three-stage training pipeline – domain text adaptation, projector alignment, multimodal VQA – produces measurable and consistent loss reduction at each stage, culminating in a system capable of image-grounded crop disease diagnosis and general agronomy question answering.

The project delivers:

- A complete, reproducible training pipeline for text, image, PDF, and audio inputs.
- A deployable FastAPI + browser-based chat application.
- Clear documentation of design choices, limitations, and improvement paths.

The immediate roadmap – expert-reviewed VQA data, extended Stage 0 training, locality-enhanced projector, and field-image evaluation – would transform the current strong prototype into a validated, production-capable agricultural diagnostic tool.

"One assistant, multiple field-friendly ways to ask for help."

References

[1] H. Liu, C. Li, Y. Li, B. Li, Y. Zhang, S. Shen, Y.J. Lee. *Improved Baselines with Visual Instruction Tuning.* arXiv:2310.03744, 2023.

[2] J. Cha, W. Kang, J. Mun, B. Roh. *Honeybee: Locality-enhanced Projector for Multimodal LLM.* arXiv:2312.06742, 2023.

[3] D. Zhu, J. Chen, X. Shen, X. Li, M. Elhoseiny. *MiniGPT-4: Enhancing Vision-Language Understanding with Advanced Large Language Models.* arXiv:2304.10592, 2023.

[4] W. Luo et al. *HyperLLaVA: Dynamic Visual Expert Tuning for Multimodal LLMs.* arXiv:2403.13kenneth, 2024.

[5] D. Hughes, M. Salathé. *An open access repository of images on plant health to enable the development of mobile disease diagnostics.* arXiv:1511.08060, 2015.

[6] Qwen Team, Alibaba Cloud. *Qwen2.5 Technical Report.* arXiv:2412.15115, 2024.